Riya Varyani
D20A / 61

Experiment: 8

AIM: Implement the Blockchain platform Ganache

Theory:

1. Introduction to Ganache

Ganache is a personal blockchain used for Ethereum development and testing. It allows developers to simulate a real blockchain environment locally without any cost or risk.

Ganache is part of the Truffle Suite and is widely used for building, testing, and debugging decentralized applications (DApps).

2. Features of Ganache

- **Instant Block Mining**
  Transactions are processed immediately without delays.
- **Pre-funded Accounts**
  Provides multiple test accounts with virtual Ether.
- **Transaction Monitoring**
  Displays complete transaction history and details.
- **User-Friendly Interface**
  Graphical dashboard to observe blockchain activity such as blocks, gas usage, and balances. ● **Deterministic Environment**
  Ensures consistent results during testing and development.

3. Integration with Development Tools

a) MetaMask

MetaMask acts as a wallet and bridge to connect Ganache with applications.

b) Remix IDE

Remix IDE is used to write, compile, deploy, and interact with smart contracts.

Ganache, MetaMask, and Remix together create a complete local development environment for Ethereum-based applications.

4. Steps to Connect Ganache with MetaMask and Remix IDE

Riya Varyani
D20A / 61

Step 1: Install and Run Ganache

- Download and install Ganache from the Truffle Suite website.
- Launch the application to start a local blockchain.
- It automatically generates accounts with test Ether.

Step 2: Connect Ganache to MetaMask

- Open MetaMask and select **Custom RPC Network**.
- Enter Ganache RPC URL: http://127.0.0.1:7545
- Import Ganache accounts using their private keys.

Step 3: Connect MetaMask with Remix IDE

- Open Remix IDE in a browser.
- Go to **Deploy & Run Transactions** tab.
- Select **Injected Web3** as the environment.
- Remix will connect automatically to the MetaMask account linked with Ganache.

5. Creating a Smart

Contract Example Solidity

Contract pragma solidity

^0.8.0; contract

SimpleStorage { uint

public data; function

set(uint x) public { data

= x;

  }

}

This contract allows storing and updating a numeric value on the blockchain.

6. Compiling and Deploying the Contract

- Compile the contract using Remix.
- Deploy it using the connected MetaMask account.

Riya Varyani
D20A / 61

- Approve the transaction in MetaMask.
- The contract is deployed on the Ganache local blockchain.

7. Monitoring Transactions in Ganache

- View transaction details such as:
  - Transaction hash
  - Gas usage
  - Block number
- Observe account balance changes in the Ganache interface.

8. Interacting with the Smart Contract

- Use Remix interface to:
  - Call set() to update data (requires transaction) ○
  Read stored data using public variable (no gas required) ●
  Track all state changes and transactions in Ganache.

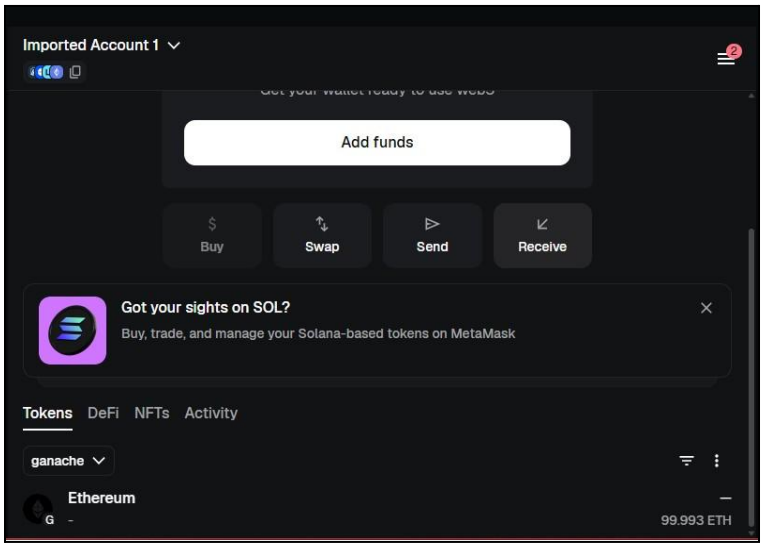
Advantages of Using Ganache

- No real cryptocurrency required
- Fast and efficient testing
- Easy debugging of smart contracts
- Complete control over blockchain environment

CODE:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract PersonalHealthRecord {

   struct Record {
string ipfsHash;
string recordType;
uint256 timestamp;
address addedBy;
   }

   struct Patient
{        string
name;        uint
age;        bool
exists;
   }

   mapping(address => Patient) public
patients;     mapping(address => Record[])
private records;     mapping(address =>
mapping(address => bool)) public
accessPermission;

   event PatientRegistered(address patient);
event RecordAdded(address patient, string
ipfsHash);      event AccessGranted(address
patient, address doctor);          event
AccessRevoked(address patient, address
doctor);

   modifier onlyPatient() {
require(patients[msg.sender].exists,
"Patient not registered");
      _;
   }

   function registerPatient(string memory
_name, uint _age) public {
require(!patients[msg.sender].exists,
"Already registered");

      patients[msg.sender] = Patient(_name,
_age, true);
```

Riya Varyani
D20A / 61



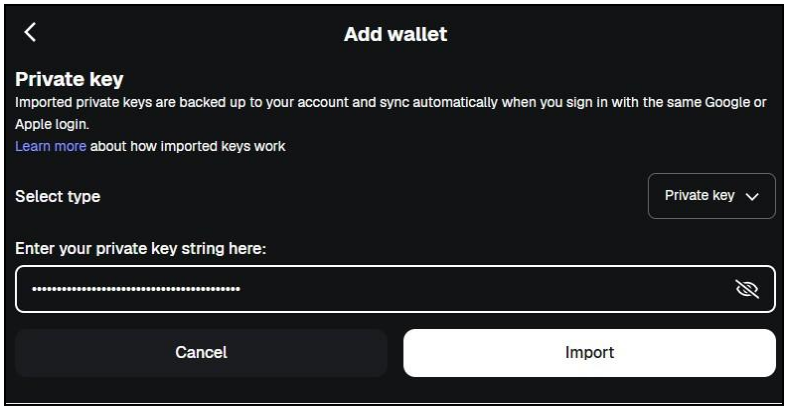

```
        emit PatientRegistered(msg.sender);
    }

    function addRecord(        string
memory _ipfsHash,        string
memory _recordType
    ) public onlyPatient {

        records[msg.sender].push(
Record({                ipfsHash: _ipfsHash,
recordType: _recordType,
timestamp: block.timestamp,
addedBy: msg.sender
            })
        );

        emit RecordAdded(msg.sender, _ipfsHash);
    }
```

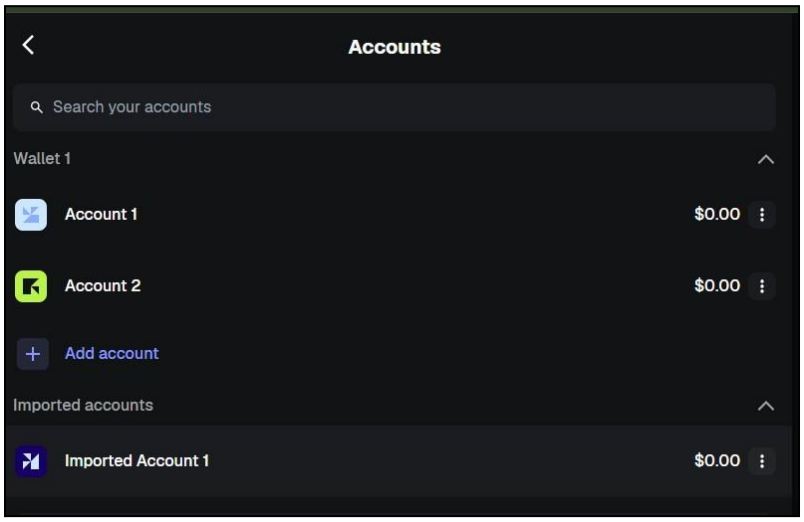
OUTPUT : Metamask:

```
function
grantAccess(address
doctor) public onlyPatient {
accessPermission[msg.se
nder][doctor] = true;
emit
AccessGranted(msg.
sender, doctor);
    }

    function revokeAccess(address doctor)
public onlyPatient {
accessPermission[msg.sender][doctor] =
false;        emit
AccessRevoked(msg.sender, doctor);
    }

    function getRecords(address
patient)        public        view
returns (Record[] memory)
    {        require(            patient ==
msg.sender ||
accessPermission[patient][msg.sen
der],
            "Access denied"
        );

        return records[patient];
    }
}
```

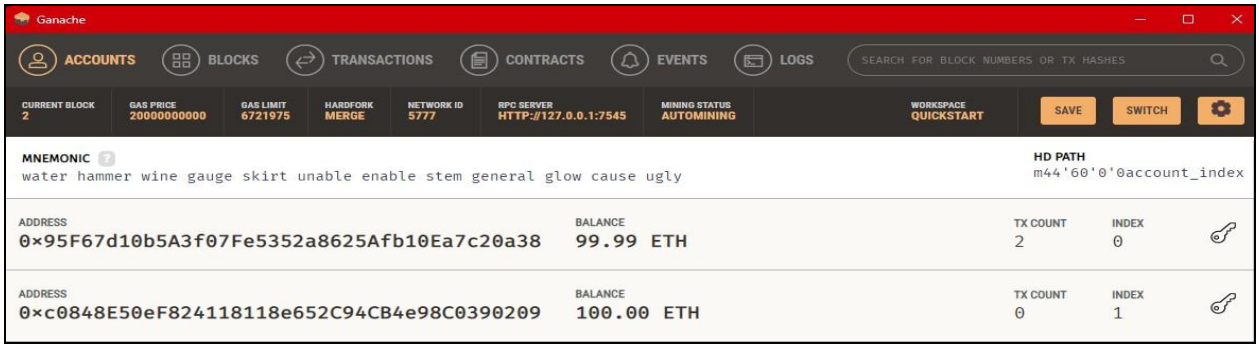
Added the wallet by giving the private key of my wallet:

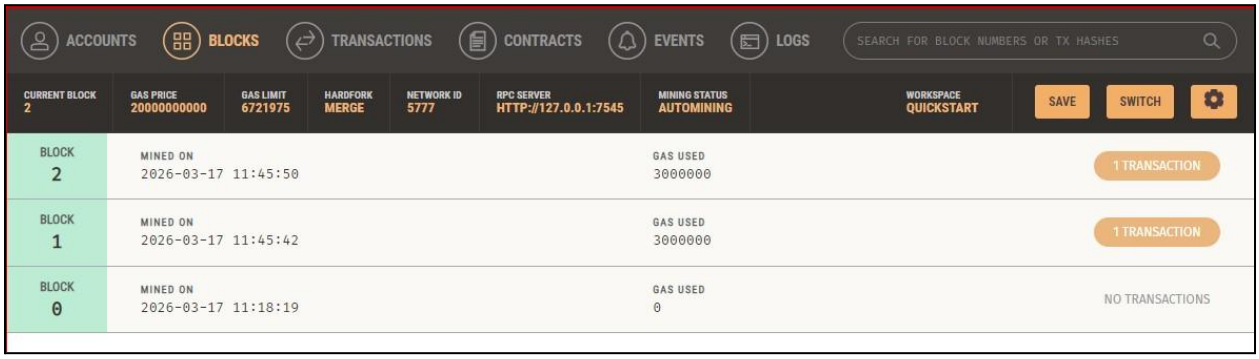
Riya Varyani
D20A / 61



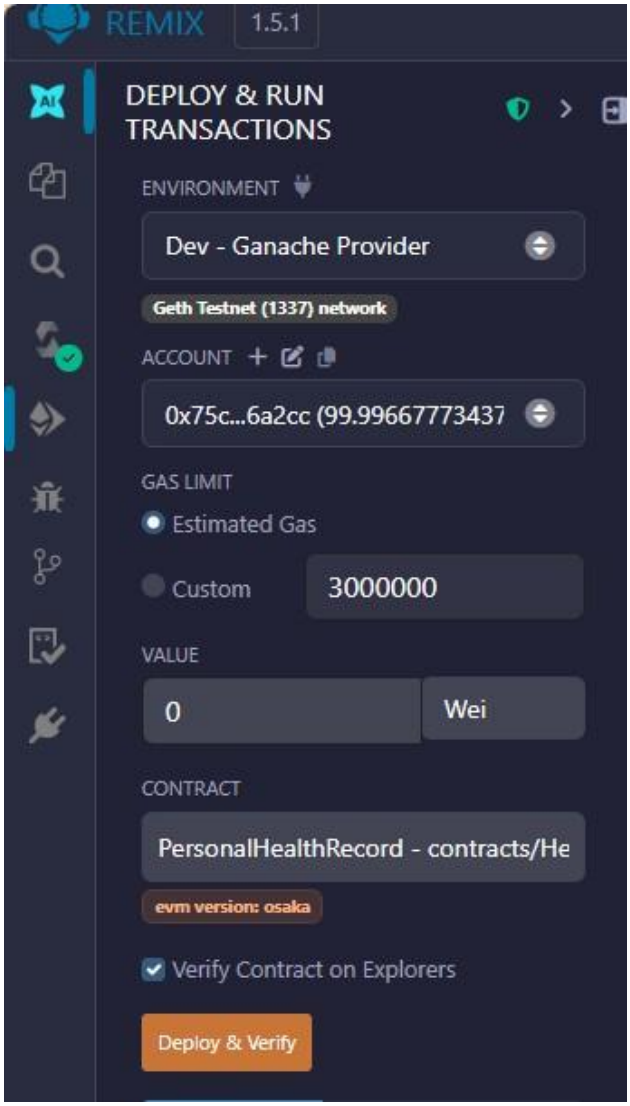


Riya Varyani
D20A / 61





Riya Varyani
D20A / 61



Conclusion:

Ganache provides a powerful local blockchain environment for testing smart contracts. When combined with MetaMask and Remix IDE, it enables developers to efficiently build, deploy, and test decentralized applications in a safe and controlled setup.